

Creating the 🌀 tempest

a cursed new take on declarative Neovim configuration

Who am I?



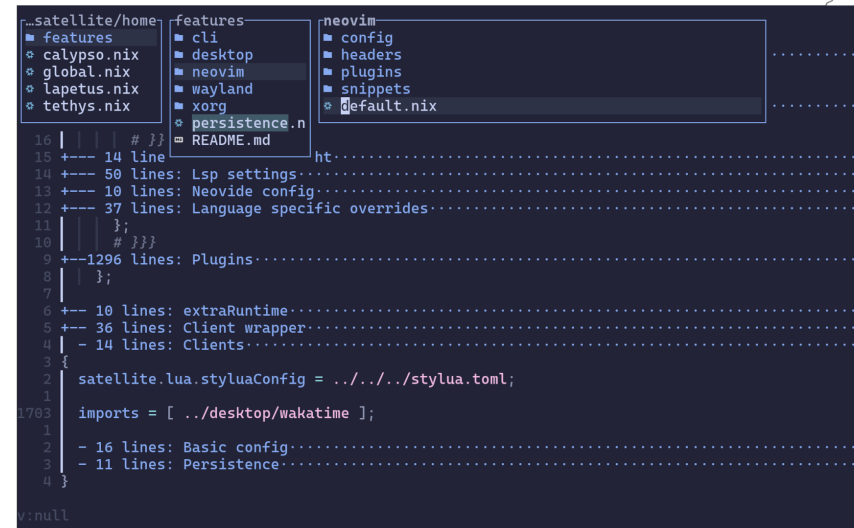
- Math student

Who am I?

- Math student
- I love 
- I use  **neovim** for everything:
 - note taking + homework ($\text{\LaTeX}$)
 - my job ( programming)
 - personal projects (, , etc)

What is neovim ?

- terminal text editor
- very extensible
-  Vimscrip or  Lua



The screenshot shows the Neovim editor interface. On the left, a file explorer displays the directory structure of a project named 'satellite/home'. The 'features' directory is expanded, showing files like 'calypso.nix', 'global.nix', 'lapetus.nix', 'tethys.nix', and 'persistence.n'. The 'neovim' directory is also expanded, showing 'config', 'headers', 'plugins', 'snippets', and 'default.nix'. The main editor window displays the contents of 'default.nix', which is a Lua configuration file. The file contains various settings and imports, including 'satellite.lua.styluaConfig', 'imports', and 'Basic config'. The editor shows line numbers on the left and right margins, and a status bar at the bottom indicates the current file is 'v:null'.

Why configure neovim using nix?

1. Access to all the options already defined using nix
2. Collocation of plugins and their nix dependencies

Why configure neovim using nix?

1. Access to all the options already defined using nix
2. Collocation of plugins and their nix dependencies

```
rust-tools = {  
  package = "simrat39/rust-tools.nvim";  
  dependencies.nix = [  
    pkgs.rust-analyzer  
    pkgs.rustfmt  
  ];  
};
```

Existing approaches

1. Install plugins using  neovim + configure using  lua
 -  `config.lib.file.mkOutOfStoreSymlink`

Existing approaches

1. Install plugins using  neovim + configure using  lua
 -  `config.lib.file.mkOutOfStoreSymlink`
2. Install plugins using  nix + configure using  lua
 - built-in  nixos module
 -  `BirdeeHub/nixCats-nvim`

Existing approaches

1. Install plugins using  neovim + configure using  lua
 -  `config.lib.file.mkOutOfStoreSymlink`
2. Install plugins using  nix + configure using  lua
 - built-in  nixos module
 -  BirdeeHub/nixCats-nvim
3. Try to do as much as possible using 
 -  nix-community/nixvim

**Can we have our cake
and eat it too?**

prescientmoon/tempest (soon)

- DSL that allows writing  lua as  nix

prescientmoon/tempest (soon)

- DSL that allows writing  lua as  nix
- Can be incrementally used next to traditional configs
- Plugins can be installed in any way
- Can even be used for non- neovim -based apps

(a note on examples)

 $\Rightarrow$  (0) – **primitives**

- Think of **builtins.toJSON** & friends

$\Rightarrow$ (0) – primitives

- Think of `builtins.toJSON` & friends
- Example usage:

```
lastplace = {  
  package = "ethanolz/nvim-lastplace";  
  opts.lastplace_ignore_buftype = [  
    "quickfix" "nofile" "help"  
  ];  
};
```



(1-2) – a challenge



```
{  
  "hrsh7th/nvim-cmp",  
  dependencies = { ... },  
  config = require("satellite.cmp"),  
}
```

$\Rightarrow$ (1) – variables

1. `<nixpkgs>` is syntax sugar for

```
builtins.findFile builtins.nixPath "nixpkgs"
```

$\Rightarrow$ (1) – variables

1. `<nixpkgs>` is syntax sugar for

```
builtins.findFile builtins.nixPath "nixpkgs"
```

2. `builtins.something` is always syntax sugar for `__something`

$\Rightarrow$ (1) – variables

1. `<nixpkgs>` is syntax sugar for

```
builtins.findFile builtins.nixPath "nixpkgs"
```

2. `builtins.something` is always syntax sugar for `__something`

3. We can shadow the builtin `__findFile` so that `<require>` in 
translates to `require` in 

$\Rightarrow$ (2) – function application

1. Normally, treating objects like functions make  sad

```
{ } "hello"  
# ^ attempt to call something which is not a function but a set
```

$\Rightarrow$ (2) – function application

1. Normally, treating objects like functions make  sad

```
{ } "hello"  
# ^ attempt to call something which is not a function but a set
```

2. If the object has a property called `__functor`, then  will call that instead:

```
{ __functor = self: hello: "${hello} world!"; } "hello"  
# hello world!
```

$\Rightarrow$ (2) – function application

1. Normally, treating objects like functions make  sad

```
{ } "hello"  
# ^ attempt to call something which is not a function but a set
```

2. If the object has a property called `__functor`, then  will call that instead:

```
{ __functor = self: hello: "${hello} world!"; } "hello"  
# hello world!
```

3. Hooray, we can make our  objects callable!

- `<require> "satellite.cmp"` $\Rightarrow$ `require("satellite.cmp")`
- `<print> [ "foo" <bar> ]` $\Rightarrow$ `print("foo", bar)`



(3): the dot – a new foe



```
{  
  "hrsh7th/nvim-cmp",  
  dependencies = { ... },  
  config = require("satellite.cmp").config,  
  init = require("satellite.cmp").init,  
}
```

 $\Rightarrow$  **(3) – property access**

-  has path types, which we do not yet use

$\Rightarrow$ (3) – **property access**

-  has path types, which we do not yet use
- Solution: allow calling  objects with paths as arguments to generate property access
 - ▶ `<require> "satellite.cmp" /config`
 $\Rightarrow$ `require("satellite.cmp").config`

$\Rightarrow$ (3) – **property access**

-  has path types, which we do not yet use
- Solution: allow calling  objects with paths as arguments to generate property access
 - ▶ `<require> "satellite.cmp" /config`
 $\Rightarrow$ `require("satellite.cmp").config`
- For convenience, allow embedding this inside our `<var>` syntax:
 - ▶ `<vim/inspect> { hello = "world"; }`
 $\Rightarrow$ `vim.inspect({ hello = "world" })`



(4) – the final boss?



```
{  
  "hrsh7th/nvim-cmp",  
  dependencies = { ... },  
  init = function()  
    vim.keymap.set(...)   
  end  
}
```

 $\Rightarrow$  **(4) – functions**

- no general way to encode  functions as  functions

$\Rightarrow$ (4) – functions

- no general way to encode  functions as  functions
- generate argument name $\Rightarrow$ pass it to the given function $\Rightarrow$ expect a  object

$\Rightarrow$ (4) – functions

- no general way to encode  functions as  functions
- generate argument name $\Rightarrow$ pass it to the given function $\Rightarrow$ expect a  object
 - `(_: <vim/keymap/set> [...])`
 $\Rightarrow$ `function() vim.keymap.set(...) end`

$\Rightarrow$ (4) – functions

- no general way to encode  functions as  functions
- generate argument name $\Rightarrow$ pass it to the given function $\Rightarrow$ expect a  object

▸ `(_: <vim/keymap/set> [...])`

$\Rightarrow$ `function() vim.keymap.set(...) end`

▸ `(who: [ "hello" who ])`

$\Rightarrow$ `function(a) return { "hello", a } end`

 $\Rightarrow$  (5) – **blocks**

- we don't always want to return from functions
- we might want to execute more than one action in a sequence



Introducing, the **block** helper!

```
who: block [  
  (<print> "hello")  
  (<print> who)  
]
```



```
function(a)  
  print("hello")  
  print(a)  
end
```



(6) – mutation

$\Rightarrow$ (6) – mutation

- objects found inside blocks are treated as variable assignments:

```
block {  
    vim.opt = {  
        wrap = false;  
        undofile = true;  
    };  
}
```

turns into

```
vim.opt.wrap = false  
vim.opt.undofile = true
```

⇒ neovim (0) – keybinds

We are allowed to create keybinds while inside a block:

```
block {  
  keys = {  
    mode = "iv";  
    mapping = "<f10>";  
    action = "<esc>";  
    desc = "Exit insert mode";  
  };  
}
```

 $\Rightarrow$  neovim (1) – autocmds

Autocmds are 's event listeners.

⇒ neovim (1) – autocmds

Autocmds are 's event listeners.

```
block {  
  autocmds = {  
    group = "UserNixSettings";  
    event = "FileType";  
    pattern = "nix";  
    action = _: <print> "^~^ Nix file detected ^~^";  
  };  
}
```



neovim (2) – context

```
block {  
  autocmds = {  
    group = "UserNixSettings";  
    event = "FileType";  
    pattern = "nix";  
    action.keys = {  
      mapping = "<leader>lg";  
      action = ...;  
    };  
  };  
}
```

Final thoughts

- We now have a general yet concise way to write  in 

Final thoughts

- We now have a general yet concise way to write  in 
- Does it work?

Final thoughts

- We now have a general yet concise way to write  in 
- Does it work?
- Should you use ?